Grand oral spé NSI : les échecs comme cas d'école de l'algorithmique et de l'IA

Tu peux construire un Grand Oral NSI sur les échecs en partant d'une question précise : comment choisir un coup sans explorer toutes les continuations ? Tu disposes alors d'un exemple pour expliquer la récursivité, les arbres et l'élagage, avant de présenter une autre approche avec AlphaZero.

Ce guide est actualisé pour la préparation en 2026. Il rassemble des éléments à sélectionner ; essayer de tout présenter rendrait ton raisonnement difficile à suivre.

Le format de l'épreuve

Le ministère décrit vingt minutes de préparation, puis dix minutes de présentation et dix minutes d'échange. Le jury choisit une de tes deux questions. Tu peux préparer un support pendant le temps de préparation et l'utiliser dans les conditions indiquées par le ministère. L'ancienne phase distincte de cinq minutes sur l'orientation n'appartient plus à ce format.

Appuie la question sur tes enseignements de spécialité et vérifie ce lien avec ton professeur. Le but est de construire une réponse argumentée, pas de réciter un catalogue de technologies ni de garantir un nombre de points au bac.

Une problématique et un plan de dix minutes

Question possible : « Comment l'élagage alpha-bêta réduit-il le calcul nécessaire pour choisir un coup ? »

Durée indicative	Développement
1 min	Une position, plusieurs choix et une question
3 min	Petit arbre, scores des feuilles et minimax
3 min	Même arbre, bornes alpha/bêta et coupure
2 min	Coût, ordre des coups et limites de profondeur
1 min	Réponse et ouverture vers l'apprentissage

Ces durées sont des repères d'entraînement. Tu peux déplacer du temps si une démonstration le demande, en gardant dix minutes pour ta présentation. Les dix minutes suivantes sont un échange avec le jury, pas la suite d'un exposé de vingt minutes.

Un minimax autonome en Python

Pour montrer l'algorithme sans devoir programmer toutes les règles des échecs, utilise un arbre numérique. Les feuilles sont des nombres, les nœuds internes des listes. MAX cherche le plus grand score et MIN le plus petit.

```
def minimax(arbre, maximise=True):
    if isinstance(arbre, (int, float)):
        return arbre
    if not arbre:
        raise ValueError("Un nœud interne doit avoir un enfant")
    valeurs = [minimax(enfant, not maximise) for enfant in arbre]
    return max(valeurs) if maximise else min(valeurs)

arbre = [[3, 5], [2, 9], [4, 1]]
print(minimax(arbre))  # 3
```

Chaque liste du deuxième niveau correspond à un choix de MIN : ses valeurs sont 3, 2 et 1. MAX choisit ensuite 3. Tu peux refaire cette remontée à la main avant d'expliquer le code.

Cet exemple est exécutable avec Python seul. Il illustre la décision sur un arbre donné ; il ne génère pas des coups légaux et n'est pas un moteur d'échecs.

Ajouter alpha-bêta

```
def alpha_beta(arbre, alpha=float('-inf'), beta=float('inf'),
               maximise=True):
    if isinstance(arbre, (int, float)):
        return arbre
    if not arbre:
        raise ValueError("Un nœud interne doit avoir un enfant")
    valeur = float('-inf') if maximise else float('inf')
    for enfant in arbre:
        score = alpha_beta(enfant, alpha, beta, not maximise)
        if maximise:
            valeur = max(valeur, score)
            alpha = max(alpha, valeur)
        else:
            valeur = min(valeur, score)
            beta = min(beta, valeur)
        if alpha >= beta:
            break
    return valeur

print(alpha_beta([[3, 5], [2, 9], [4, 1]]))  # 3
```

Après la première branche, MAX peut obtenir 3. Dans la deuxième, MIN dispose déjà de 2 : explorer 9 ne rendrait pas cette branche préférable pour MAX. La coupure conserve le résultat du minimax pour ce même arbre et ces mêmes évaluations.

L'ordre des enfants influence le nombre de branches visitées. Il ne change pas la valeur obtenue par une recherche correcte sur l'arbre complet considéré.

Expliquer le coût sans transformer une estimation en comptage exact

Sur un arbre régulier de branchement b et de profondeur d, le nombre de feuilles est b puissance d. Le coût du minimax naïf est de l'ordre de $O(b^d)$. Les échecs réels n'ont pas exactement 35 coups légaux à chaque position : 35 peut servir d'hypothèse de calcul.

Avec un ordre idéal des coups, l'ordre de grandeur usuel d'alpha-bêta est $O(b^{d/2})$; dans le pire cas, il reste $O(b^d)$. Pour b = 35 et d = 6, comparer les puissances donne $35^6 = 1\,838\,265\,625$ et $35^3 = 42\,875$. Leur rapport est 42 875. Ce n'est pas une promesse de gain réel identique, ni le nombre exact de tous les appels récursifs.

Une recherche arrêtée à une profondeur donnée dépend de son évaluation. Elle ne prouve pas que le coup retenu gagne contre toute défense jusqu'à la fin de la partie.

Représenter une position

Un tableau peut coder les pièces par case. Un bitboard représente un ensemble de cases avec les bits d'un entier. Par convention a1 = 0 et h8 = 63, les pions blancs initiaux occupent les bits 8 à 15 :

```
pions_blancs = 0xFF00
case = 12  # e2 selon cette convention
print(bool(pions_blancs & (1 << case)))  # True
```

Ce nombre ne code pas toute la position : il faut aussi les autres pièces, le trait, les droits de roque et l'état de la prise en passant. L'historique peut compter pour certaines règles de nulle.

Les transpositions sont des états atteints par des suites différentes. Une table de transposition demande une clé adaptée et doit distinguer une valeur exacte d'une borne issue d'une coupure. Un hash n'est pas une compression sans perte : des collisions sont possibles.

Des données de parties : PGN et SQL

La spécification PGN d'Edwards (1994) définit les en-têtes et le texte des coups. Une expression régulière qui extrait quelques balises ne constitue pas un parseur complet : commentaires, variantes et échappements demandent davantage de traitement.

Voici une partie fictive courte, sans prétendre reproduire Deep Blue-Kasparov :

```
[Event "Exemple pédagogique"]
[Site "?"]
[Date "2026.01.01"]
[Round "?"]
[White "Blancs"]
[Black "Noirs"]
[Result "0-1"]

1. f3 e5 2. g4 Qh4# 0-1
```

Le résultat `0-1` figure dans l'en-tête et à la fin du texte. Garde cette cohérence dans ton propre fichier.

Une table de parties peut ensuite servir à compter des résultats par ouverture. Mais un taux de victoire observé ne prouve pas qu'une ouverture est causalement meilleure : niveau, sélection des joueurs et cadence peuvent différer.

AlphaZero : apprendre et rechercher

Le préprint de Silver et collègues (2017) associe un réseau à deux sorties - politique et valeur - à une recherche MCTS. L'auto-jeu fournit les données d'entraînement. Le système reçoit notamment les règles, une représentation et une procédure conçues par les chercheurs ; il n'apprend pas sans cadre humain.

Le protocole de 2017 contre Stockfish 8 rapporte 28 victoires, 72 nulles, aucune défaite, à une minute par coup. Les quatre heures sont un repère sur la courbe d'apprentissage dans ce dispositif, pas la durée totale de tous les entraînements. Les performances par seconde dépendent du matériel et du protocole.

Le Stockfish actuel ne se réduit pas à une formule entièrement écrite à la main : la version 12 a adopté NNUE en 2020. Une recherche alpha-bêta et une évaluation apprise peuvent donc coexister.

AlphaZero n'est pas AlphaFold. Si tu veux prolonger vers la science, utilise le sujet AlphaZero pour distinguer les tâches et les chronologies.

Questions à préparer

- Pourquoi MAX et MIN alternent-ils ? Explique l'hypothèse d'un adversaire qui choisit sa meilleure réponse dans ce modèle.
- Pourquoi une coupure est-elle possible ? Refais les bornes sur le petit arbre.
- Que change l'ordre des coups ? Le coût observé, avec le même résultat pour la recherche considérée.
- Une table est-elle toujours réutilisable ? Vérifie l'état, la profondeur et la nature de l'information mémorisée.
- Un moteur fort a-t-il résolu les échecs ? Non : performance en parties et preuve de la valeur initiale sont distinctes.
- Pourquoi le nombre de Shannon ne compte-t-il pas des positions ? Il estime un ordre de grandeur de suites de jeu dans les hypothèses de Shannon (1950).

Préparer ton support

Choisis un arbre assez petit pour expliquer chaque feuille, le code correspondant et une conclusion délimitée. Vérifie les calculs, les exemples et les sources. Prévois une présentation compréhensible sans exécuter une application devant le jury.

Pour les développements complémentaires, le guide NSI détaillé reprend minimax et alpha-bêta ; le guide Maths traite les ordres de grandeur et les probabilités.

Sources

- Ministère de l'Éducation nationale (2026), *Baccalauréat : comment se passe le Grand oral ?* Format officiel.
- Russell, S. et Norvig, P. (s. d.), *Adversarial Search*, projet de chapitre d'Artificial Intelligence: A Modern Approach, Berkeley. Texte.
- Edwards, S. J. (1994), *Portable Game Notation Specification and Implementation Guide*. Spécification.
- Silver, D. et al. (2017), *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, préprint arXiv. Article.
- Équipe Stockfish (2020), *Stockfish 12*. Annonce NNUE.
- Shannon, C. E. (1950), *Programming a Computer for Playing Chess*, Philosophical Magazine, 41, 256-275. Article.